

Energy-Guided Sampling for Controllable Text Generation

From Langevin Dynamics to Amortized Inference

Sarthak Singh

Institut für Maschinelle Sprachverarbeitung, Universität Stuttgart

Examiners: Prof. Dr. Ngoc Thang Vu Dr. Antje Schweitzer

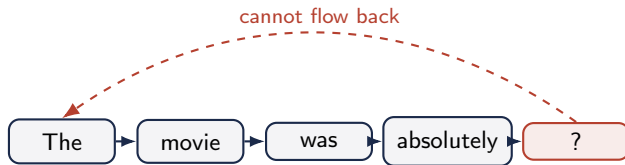
Supervisor: Prof. Dr. Ngoc Thang Vu

M.Sc. Computational Linguistics

Thesis Defense

The problem autoregressive models cannot solve natively

- Autoregressive generation: one token at a time, left to right.
- A token is fixed the moment it is emitted.
No mechanism to revise an earlier token in light of a later one.
- Global properties (sentiment, factual constraint) live in the *whole* sequence.



Control means steering the *completed* object. That is a global problem.

The energy-based promise, and its one assumption

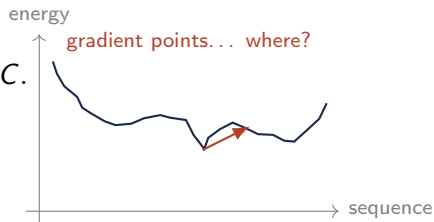
Treat the frozen likelihood as an **energy** over sequences and sample from it:

$$p(\mathbf{x}) \propto \exp(-E(\mathbf{x})), \quad E = -\log p_{\text{LM}} + \lambda C.$$

- Add any constraint C at inference time.
- One frozen model, any objective.
Plug and play.

The untested assumption:

the gradient of the frozen likelihood is a usable search direction on discrete text.



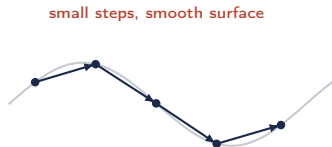
A jagged landscape: the local slope need not lead anywhere good.

The machinery: Langevin dynamics

Sample $p(\mathbf{s}) \propto e^{-E(\mathbf{s})}$ with a gradient-guided chain:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \frac{\epsilon_t}{2} \nabla_{\mathbf{s}} \log p(\mathbf{s}_t) + \sqrt{\epsilon_t} \boldsymbol{\xi}_t$$

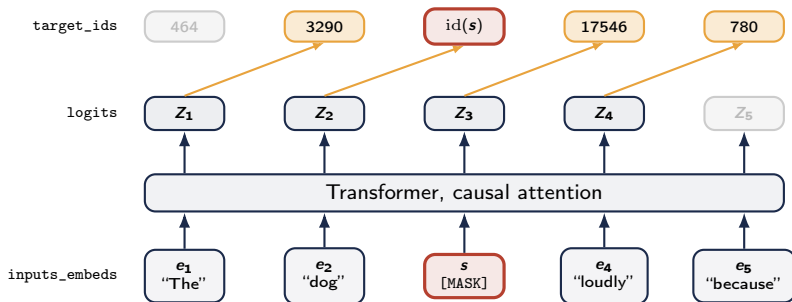
- **Drift:** uphill.
- **Diffusion:** a *sample*, not a mode.
- Needs a **Lipschitz** drift.



Text is *discrete*: there is no small step.

Every method here forces this update onto discrete tokens.

Scoring a sequence: two arrays, offset by one



```
logits = out.logits[:, :-1, :]  
targets = target_ids[:, 1:]
```

$$E(\mathbf{x}) = -\sum_t \log \text{softmax}(\mathbf{Z}_t)[\text{id}_{t+1}]$$

Position 3 is masked: its embedding is the soft token s we optimize.
The same sequence enters twice: as continuous vectors, and as integers.

Research questions

RQ1 Does the *input-embedding* gradient give a usable proposal direction? If not, what does?

RQ2 What does the Metropolis–Hastings correction do in each sampler?

RQ3a Does a GFlowNet on this reward learn a high-reward policy?

RQ3b Does its tuned energy become more navigable?

E (extension) Can an additive constraint steer generation here?

Three candidate explanations

Suppose the gradient does not help. *Three* things could be at fault.

	What is at fault	Decided by
A	the target : the energy itself	gradient-free search, same energy
B	the model : how it was trained	a different derivative, same weights
C	the parameterization : which derivative	a ladder of proposals, one chain

Everything that follows is that elimination, in that order.

The setup: one task, five energies, one metric

Task. Masked-token recovery: corrupt tokens, ask the sampler to put them back.

200 sequences per configuration, paired seeds across arms.

Metric. Mean KL to the ground-truth fill. Floor of 0.

Five energies.

- **GPT-2 Large**, fine-tuned. The reference.
- **Three GFlowNet-tuned** variants of it.
- **Llama-3 8B**. Cross-architecture.

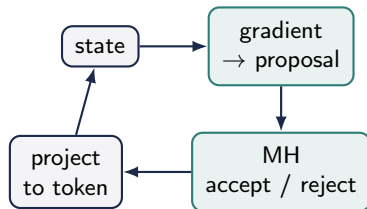
145 configurations. Equivalence margin fixed in advance at 0.327.

Two samplers, implemented faithfully

- **DLS** (discrete): stays in token space.
- **CLS** (continuous): relaxes into embedding space, projects back.
- **Metropolis–Hastings** correction, applied exactly.
- Step size by **oracle calibration**.

DLS scores a candidate v against x_i , with $\mathbf{g} = \nabla_{\mathbf{e}_i} \log p_{\text{LM}}$:

$$\log q(v \mid x_i) \propto \underbrace{\frac{1}{2} \mathbf{g}^\top (\mathbf{e}(v) - \mathbf{e}(x_i))}_{\text{gradient alignment}} - \underbrace{\frac{1}{2\alpha} \|\mathbf{e}(v) - \mathbf{e}(x_i)\|^2}_{\text{distance}}$$



the step the field skips

A fixed annealed schedule, never stopped early.

This $\mathbf{g}$, the input-embedding gradient, is the object under test.

First: what does the calibrated proposal actually look like?

Measure the entropy of the proposal the grid draws from.

Flagship configuration

step 0: 10.8248 nats

floor over 50 steps: 10.28

uniform ceiling $\log |V| = 10.8249$

Gradient term: ≈ 0.009 nats inside a 10.82-nat budget.

The calibrated proposal is numerically uniform.

Why this comes first

Two proposals both within 10^{-4} nats of uniform are two draws from the *same* distribution.

A perfectly informative gradient would have produced the same null here.

So a 5×5 sweep over step size and temperature re-tests it where the gradient *does* shape the proposal.

The central result: gradient vs. a norm-matched random direction

Isolate the gradient's *direction* from its *magnitude*.

Paired contrast, $n = 1000$

last iterate	+0.133	$[-0.063, +0.326]$
chain mean	+0.136	$[-0.012, +0.287]$
chain minimum	+0.033	$[-0.079, +0.145]$

All inside the ± 0.327 margin: equivalence certified.

In the sweep, recovery never exceeds 2% in any of the 25 cells.

The gradient direction is equivalent to noise of the same norm.

Energy	policy	random
GPT-2 Large	6.474	6.430
GFN lb0-500	6.009	6.001
GFN lb0-2000	6.698	6.552
GFN lb1-500	6.008	5.338

Final KL, lower better.

Hypothesis A: is the target at fault?

Score the *same* energy without ever taking its gradient:

Method (no gradient of the energy)	mean KL
Ground-truth fill (floor)	0.00
Corrupted token left in place	9.14
Best Langevin configuration (<i>uses the gradient</i>)	~ 6.4
Gradient-free Gibbs sampler	6.69
Top- k rescore, one forward pass	4.43

A is rejected: the energy is searchable, just not by this derivative.

Why, part 1: the linearization fails before the first move

Linearization means replacing the true change a substitution causes by its first-order Taylor term, which is exactly what the proposal ranks by:

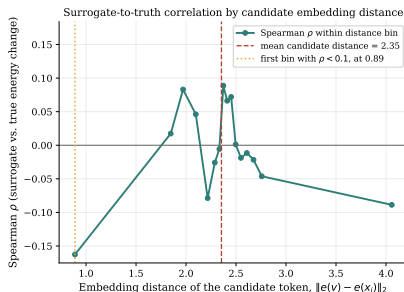
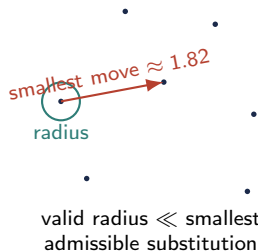
$$\hat{\Delta}(v) = \mathbf{g}^\top (\mathbf{e}(v) - \mathbf{e}(x_i))$$

$$\Delta_{\text{true}}(v) = \log p(\mathbf{x}_{i \rightarrow v}) - \log p(\mathbf{x})$$

Correlate the two over $n = 400,000$ candidate-sequence pairs.

$|\rho| < 0.06$ on *all five* energies.

The gradient is inapplicable, not merely imprecise.



Why, part 2: the correction breaks the continuous sampler

Decompose the acceptance ratio for boundary-crossing moves:

The two terms

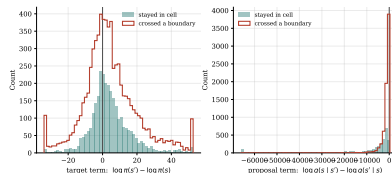
target log-ratio = +4.60 (good moves)
proposal log-ratio = -1325 (kills them)

- The projected drift is not Lipschitz across a cell boundary.
- CLS accepts within-cell moves $\approx 0\%$; DLS, 100%.

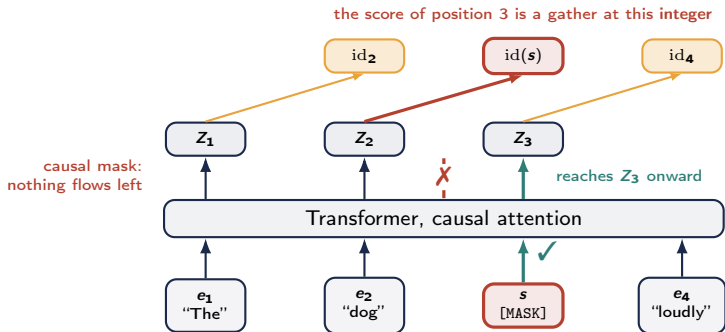
The exact correction disables CLS; it rescues DLS.

Exact for the proposal it corrects. What fails is the regularity underneath.

Metropolis-Hastings acceptance ratio, target and proposal terms, by whether the proposal crossed a Voronoi cell boundary



Back to the two arrays: what the gradient cannot reach



z_2 was built from e_1, e_2 alone. The only thing position 3 contributes to *its own* score is that integer, so the derivative is exactly 0.

future term, what it had
 $g^\top (e(v) - e(x_i))$

+

self term, read off the logits
 $\log p(v \mid x_{<i})$

Hypothesis B: same model, same sampler, different derivative

Relax the token indicator $\mathbf{z}_i$ in *both* of its roles:

$$\frac{\partial \tilde{L}}{\partial \mathbf{z}_i[v]} = \underbrace{\log p(v \mid \mathbf{x}_{<i})}_{\text{self, discarded}} + \underbrace{\mathbf{g}^\top \mathbf{e}(v)}_{\text{future}}$$

Same weights, same sampler, same energy.
Only the derivative changes.

Surrogate	ρ (near)	exact
input embedding	0.03	0.0%
token indicator	0.73	40.0%

Llama-3 8B reaches 41.0% where every input-embedding run gives 0.0%.

$$0 \rightarrow 40\%$$

B rejected in its strong form

At the calibrated $T = 5$ even this falls to 2%: the **temperature** was the binding constraint.

The exact case: the final-token gradient is zero

Under causal attention with `inputs_embeds`, the gradient of the sequence log-likelihood w.r.t. the *final* token's input embedding is

$$\nabla_{\mathbf{e}_L} \log p_{\text{LM}}(\mathbf{x}) = 0 \quad (\text{exactly}),$$

while the energy *difference* between candidate final tokens is exactly the model's conditional log-ratio. Energy maximally usable; gradient provably useless.

Position	LM grad norm
final token	0.0000
second-to-last	> 0
middle	> 0

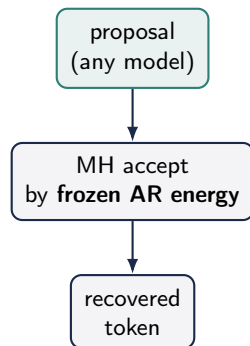
Langevin recovers 0% at all three positions; exact-energy methods 18–55%.

Hypothesis C: the conditioning ladder

One Metropolis chain, one exact AR energy. *Only the proposal changes.*

Proposal	conditioning	exact
input-emb. gradient	none	0.0%
norm-matched random	none	0.0%
uniform draw	none	0.5%
AR conditional	left only	23.5%
SEDD (score-trained)	both sides	39.0%
RoBERTa MLM (no score obj.)	both sides	44.5%

The uniform draw reproduces the flagship Langevin sampler: KL 6.538 vs 6.541.



Gradient-free comparator: top- k rescore 33%. So 39% is a gain over 33, not over nothing.

Score training is sufficient, not necessary.

GFlowNet amortization does not escape the failure

A GFlowNet only ever *evaluates* the reward, never differentiates it.

- Three failure modes: **capacity collapse**, **length collapse**, **reward hacking**.
- Tuning moved the energy by 31–57 nats. . .
- . . . yet Langevin sampling on the tuned energy is *indistinguishable* from the untuned base.

Changing the model through this objective did not make the local surrogate useful.

The extension: does an additive constraint steer anything?

$E = -\log p_{\text{LM}} + \lambda C$, sentiment head on the frozen base.

Raw gains are swamped by a sentiment drift, so the statistic is a paired contrast that cancels it.

Setup (continuation)	Target	Contrast
MuCoLa continuous	negative	+27.3
MuCoLa continuous	positive	+36.7
Discrete sampler (ours)	negative	0.0
Discrete sampler (ours)	positive	+0.7

On a landscape a proposal can navigate:

- Guidance moves the classifier's own verdict by +9.3 to +25.7 points.
- Transfer to an independent judge is bounded by instrument agreement, 56–80%.
- A trust region removes the fluency cost (7.1 $\rightarrow$ 11.0 nats without it).

A constraint added to a landscape no sampler can navigate is added to nothing.

Takeaways

- ❶ Not the derivative you think. The *input-embedding* gradient is equivalent to a norm-matched random direction, certified over 1000 paired sequences.
- ❷ Not the target, and not the model. The same frozen weights, differentiated in token-indicator coordinates, recover 40% and 41%.
- ❸ What buys the rest is right-context access, not a score objective: RoBERTa 44.5% > SEDD 39% > AR conditional 23.5%.
- ❹ Control follows navigation.

Differentiate the right object, or propose from the output side.

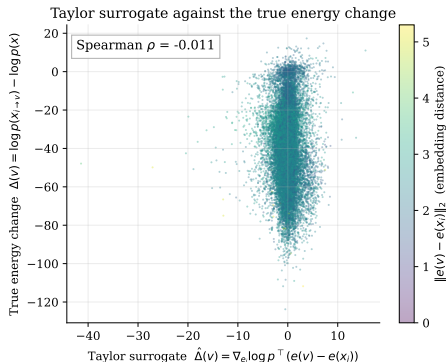
Not: evaluate rather than differentiate. The derivative was in the wrong coordinates.

Thank you.

Questions?

Backup: the linearization radius and self-term blindness

- The surrogate correlates with the *future* term ($\rho = 0.03$), never the *self* term ($\rho = -0.10$).
- Mean absolute self term 15.0 nats; future term 24.2 nats.
- The input-embedding gradient is structurally blind to the candidate's own left-context fit, which enters as a discrete index.



Backup: the Metropolis–Hastings chain

$$\alpha = \min \left\{ 1, \frac{p(\mathbf{s}') q(\mathbf{s} \mid \mathbf{s}')}{p(\mathbf{s}) q(\mathbf{s}' \mid \mathbf{s})} \right\}$$

- Target ratio: prefers good states. Proposal ratio: enforces reversibility.
- MALA needs a Lipschitz drift (Roberts and Tweedie 1996; Dalalyan 2017; Durmus and Moulines 2017).
- The projected target is piecewise constant; the differentiable pathway's drift jumps across a cell boundary, so the reverse density vanishes.

Backup: what the sampler does without the correction

- Correction off: a uniform random walk that never localizes. Mean KL *rises* from 8.765 to 9.499; the chain still changes its token on 99.1% of final-decile steps.
- So the late entropy dip is the annealing floor, not a quench: no freezing, no convergence to a local optimum.
- Correction on: the same uniform proposal becomes an independence sampler, the accept/reject filter does all the work, final KL 6.541 at a 9.98% acceptance rate.

Backup: likelihood is a poor objective, yet used everywhere

- The **likelihood trap**: the lowest-energy text is the most degenerate (Holtzman 2020, reproduced here). Within-strategy repetition correlation up to +0.91.
- Resolution: likelihood ranks single-token substitutions well (used by the KL metric and Gibbs), but maximizing absolute sequence likelihood in free generation is degenerate.
- So using it as the KL metric, the reward, and the SEDD comparison is consistent: relative-to-reference fit, not absolute likelihood as quality.

Backup: MuCoLa / COLD, and why they appear to work

- The continuous sampler follows the COLD/MuCoLa mechanism faithfully, including the correction those works typically omit.
- Without the exact correction the update, *as reimplemented and measured here*, behaves as an early-stopped biased optimizer rather than a sampler from the stated energy. That is a statement about the mechanism, not a reinterpretation of those systems' published results, whose tasks, tuning and evaluation this study did not reproduce.
- The constrained *mucola* arm is that comparison, run at the energy level (final KL under the same base model), not the task level.

Backup: task design and corpus

- Masked-token recovery on WikiText-2 validation; the base GPT-2 Large is fine-tuned on ROCStories (out-of-domain test).
- In-domain ROCStories diagnostics reproduce the null and the likelihood trap: not a domain-shift artifact.
- The KL metric compares the recovered token's next-token conditional against the ground-truth-conditioned reference; synonyms score well.
- Llama-3 is a cross-architecture control, never on a shared numerical axis. One Llama contrast is nominally significant against the gradient, but it is one of dozens uncorrected, is measured with the correction off, and does not reproduce on GPT-2 sharp cells; the supported claim is *indifference*, not anti-guidance.

Backup: why did the temperature flatten everything?

- Proposal logit =
$$\left[\underbrace{\frac{1}{2} \mathbf{g}^\top (\mathbf{e}(v) - \mathbf{e}(x_i))}_{t_2} - \underbrace{\frac{1}{2\alpha} \|\mathbf{e}(v) - \mathbf{e}(x_i)\|^2}_{t_1} \right] / T.$$
- At the calibrated cell $T = 5$ divides a spread of a few nats into a 10.82-nat entropy budget: the softmax is uniform whatever the surrogate says.
- Sharpness from the *distance* term is sharp about *staying put* ($t_2/t_1 = 1.8$); sharpness from the *surrogate* is sharp about the ranking ($t_2/t_1 = 229$). Only the second tests the direction.
- The recovering cell needs large ϵ (kills t_1) and low T together.

Backup: what exactly is the token-indicator derivative?

Relax the token indicator $\mathbf{z}_i$ from its simplex vertex, in *both* roles:

$$\tilde{L}(\mathbf{z}_i) = \sum_v \mathbf{z}_i[v] \log p(v \mid \mathbf{x}_{<i}) + L_{\text{future}}(\mathbf{E}^\top \mathbf{z}_i)$$

- First term: the current position's own log-likelihood with the discrete *target* relaxed. Second term: the suffix, with the relaxed indicator mapped through $\mathbf{E}$ to an input embedding.
- At a vertex $\tilde{L}$ equals the sequence log-likelihood, so this is a relaxation of the same object.
- It is *not* $\mathbf{E}^\top \mathbf{g}$, the ordinary one-hot input gradient through the embedding lookup: that carries the future term only. The whole difference is the self term.

Backup: why does RoBERTa beat SEDD?

- Both are bidirectional; only SEDD is score-trained. RoBERTa 44.5% vs SEDD-medium 39.0%, so the score objective is not what buys recovery.
- RoBERTa-large is bigger and trained on more text than SEDD-small/medium, and masked-token prediction *is* its native query at any noise level, whereas SEDD answers it at a schedule point.
- So the claim is an *ordering by conditioning*, not an isolation of it and not a ranking of the two models: the arms differ in scale, corpus and architecture as well, and only the tokenizer and the chain are held fixed across all of them. The SEDD arms are pilots.
- Tokenizer bridge: 99.95% of RoBERTa ids map to one GPT-2 token; no ground-truth token fell outside it. Uniform control through the same code: 0.5%, so the pipeline leaks nothing.

Backup: why a GFlowNet, and the task-comparability caveat

- A GFlowNet samples in proportion to reward (not the mode), and never differentiates it, so a gradient pathology would be invisible to it.
- It reformulates infilling as left-to-right generation under a restructured prompt; the samplers do in-place bidirectional substitution.
- The comparison is therefore run at the energy level, not the task level; the left-to-right reformulation is not adopted for the samplers because it would dissolve the revision capability under study.

Backup: how this differs from MuCoLa and COLD

Method	differentiates w.r.t.	self term	MH
MuCoLa, COLD	input embeddings $\mathbf{e}_i$	discarded	omitted
CLS, DLS (this thesis)	input embeddings $\mathbf{e}_i$	discarded	exact
Grathwohl '21, Zhang '22	the one-hot $\mathbf{z}_i$	kept	exact
token indicator (ours)	the relaxed $\mathbf{z}_i$	kept	exact

- Yes, they share the blindness, and it is not a bug in either. It follows from differentiating with respect to a coordinate that the token's own score does not pass through. Nothing else about their formulation is doing the damage.
- The gap is a borrowing. The discrete-EBM literature the DLS proposal comes from differentiates the *one-hot* input, which does carry the self term. The embedding-space text methods differentiate the input embedding, which does not. This thesis's samplers inherited the text-side choice.
- What would change it. A method that also relaxes the *target*, scoring a soft vector through the tied output matrix rather than gathering at an index, makes the self term differentiable. That is precisely what the token-indicator derivative recovers.
- Deviations that are ours, not theirs. The constraint enters as a weighted sum (0.8 : 0.2) rather than MuCoLa's Lagrangian with ϵ thresholds, and the correction is applied exactly. The comparison is at the level of the shared mechanism as reimplemented here, not a reproduction of their systems' tasks or tuning.